

Kotlin Intermédiaire

Maîtrisez les concepts avancés
pour des applications robustes
et maintenables



Extensions,
coroutines, DSL,
sérialisation, et plus

KOTLIN - INTERMEDIAIRE

Kotlin

Extensions, sealed, scope functions et
coroutines (idée) pour du Kotlin
idiomatique.

DanielCraft - Formation Kotlin

Sommaire

1. [Après les bases : Kotlin intermediaire](#)
 - [Ce que tu vas apprendre](#)
 - [A retenir](#)
2. [Collections avancees](#)
 - [Mutable vs immutable](#)
 - [A retenir](#)
3. [Fonctions d'extension](#)
 - [Pourquoi](#)
 - [A retenir](#)
4. [Sealed classes et interfaces](#)
 - [Interet](#)
 - [A retenir](#)
5. [when avance](#)
 - [Usages](#)
 - [A retenir](#)
6. [Scope functions : let, run, apply, also, with](#)
 - [Aide-memoire](#)
 - [A retenir](#)
7. [Fonctions d'ordre superieur](#)
 - [Inline \(idee\)](#)
 - [A retenir](#)
8. [Delegation](#)
 - [by lazy](#)
 - [A retenir](#)
9. [Coroutines \(idee\)](#)
 - [Concepts](#)
 - [A retenir](#)
10. [Flow \(idee\)](#)
 - [Vs LiveData / callbacks](#)
 - [A retenir](#)
11. [Null-safety avancee](#)
 - [Outils](#)
 - [A retenir](#)
12. [Kotlin Multiplatform \(idee\)](#)
 - [Idee](#)
 - [A retenir](#)
13. [Tests \(idee\)](#)
 - [Coroutines](#)
 - [A retenir](#)

14. [Mini-projet : journal d'humeur](#)

- [Exigences](#)
- [Livrables](#)
- [A retenir](#)

15. [Ce qu'il faut retenir](#)

- [Carte inter Kotlin](#)
- [Mindset](#)
- [A retenir](#)

16. [Atelier : extensions](#)

- [A retenir](#)

17. [Atelier : sealed + when](#)

- [A retenir](#)

18. [Erreurs classiques](#)

- [A retenir](#)

19. [Bonnes pratiques](#)

- [A retenir](#)

20. [Quiz Kotlin inter](#)

- [Bareme](#)
- [A retenir](#)

21. [Bravo !](#)

- [Et maintenant ?](#)
- [A retenir](#)

Après les bases : Kotlin intermédiaire



Des bases aux idiomes Kotlin.



Couverture Kotlin Intermédiaire.

Tu maîtrises null-safety, fonctions, data classes. L'**intermédiaire**, c'est **idiomes Kotlin** : extensions, sealed, scope functions, collections avancées, coroutines (idée).

> **Astuce DanielCraft** - Ecrire du Kotlin idiomatique, pas du Java traduit.

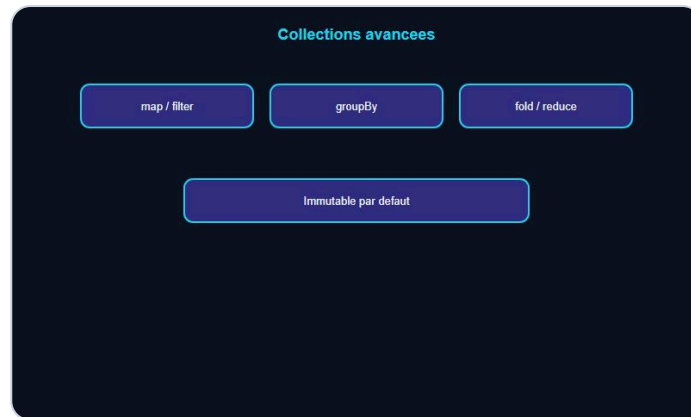
Ce que tu vas apprendre

- Extensions, sealed, when avance.
- Scope functions, higher-order.
- Coroutines / Flow (idée).
- Delegation, mini-projet.

A retenir

- Inter = expressivité + sûreté.

Collections avancées



map, filter, groupBy au quotidien.

```
val noms = listOf("Lea", "Max", "Sam")
val maj = noms.map { it.uppercase() }
val longs = noms.filter { it.length > 3 }
val parLongueur = noms.groupBy { it.length }
```

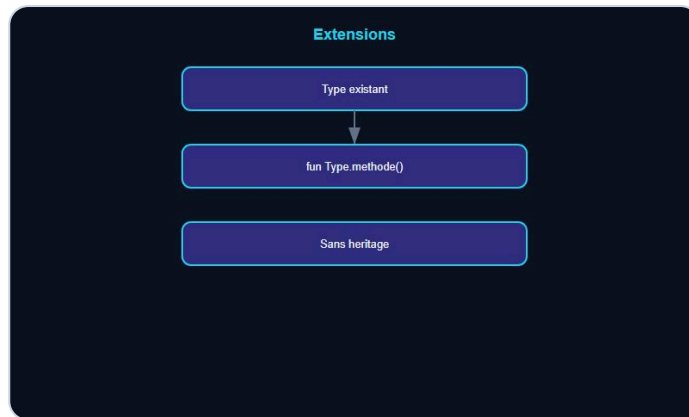
Mutable vs immutable

- `listOf` / `mutableListOf`.
- Préférer immutable en API publique.

A retenir

- map / filter / groupBy = quotidien Kotlin.

Fonctions d'extension



Enrichir un type sans heritage.

```
fun String.initiales(): String =  
    split(" ").mapNotNull { it.firstOrNull()?.uppercaseChar() }.joinToString("")  
  
println("Loic Daniel".initiales()) // LD
```

Pourquoi

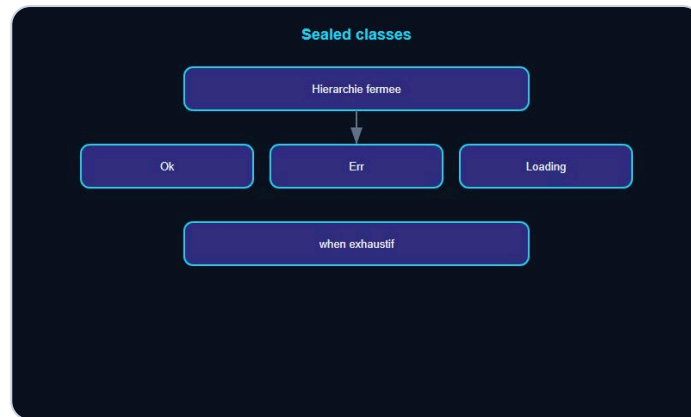
- Enrichir un type sans heritage.
- Garder les appels lisibles.

> **Piege** - Trop d'extensions = API introuvable. Namespace clair.

A retenir

- Extension = verb métier sur un type existant.

Sealed classes et interfaces



Etats fermes, when exhaustif.

```
sealed class Resultat {
    data class Ok(val data: String) : Resultat()
    data class Err(val msg: String) : Resultat()
}

fun afficher(r: Resultat) = when (r) {
    is Resultat.Ok -> r.data
    is Resultat.Err -> r.msg
}
```

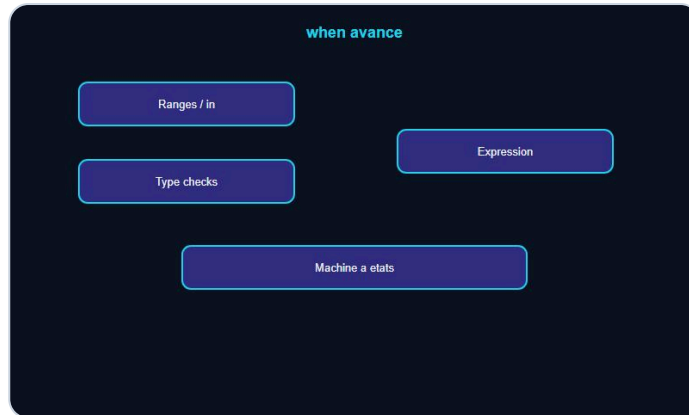
Interet

- when **exhaustif** : le compilateur verifie tous les cas.

A retenir

- Sealed = hierarchie fermée, parfaite pour etats.

when avance



when expression et machine a etats.

```
val label = when (x) {  
    in 1..10 -> "petit"  
    in 11..100 -> "moyen"  
    else -> "autre"  
}
```

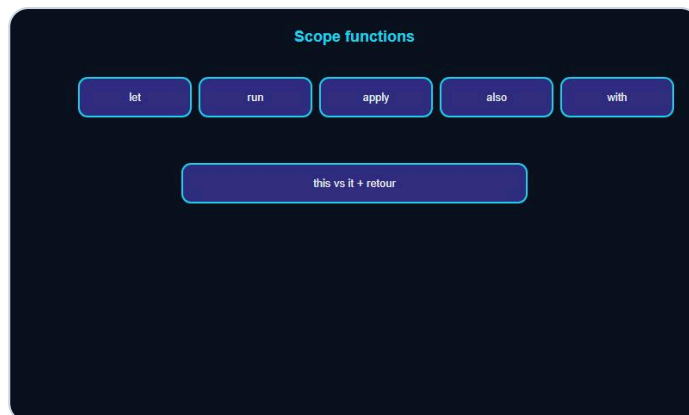
Usages

- Remplacer if cascades.
- Avec sealed : machine a etats lisible.

A retenir

- `when` = expression, pas seulement instruction.

Scope functions : let, run, apply, also, with



let, run, apply, also, with.

```
val user = User().apply {  
    name = "Lea"  
    age = 30  
}  
  
val len = name?.let { it.length } ?: 0
```

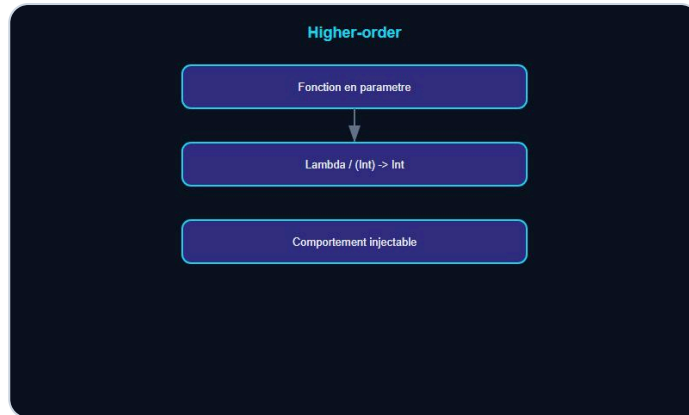
Aide-memoire

Fonction	Objet	Retour
let	it	lambda
run	this	lambda
apply	this	objet
also	it	objet

A retenir

- Choisir selon ce que tu veux retourner.

Fonctions d'ordre superieur



Passer une fonction en parametre.

```
fun operer(a: Int, b: Int, op: (Int, Int) -> Int) = op(a, b)

operer(2, 3) { x, y -> x + y }
```

Inline (idee)

- `inline` reduit le cout des lambdas dans les hot paths.

A retenir

- Passer une fonction = comportement injectable.

Delegation



by inner et by lazy.

```
interface Repo { fun find(id: Int): String? }

class CacheRepo(private val inner: Repo) : Repo by inner {
    // surcharge possible
}
```

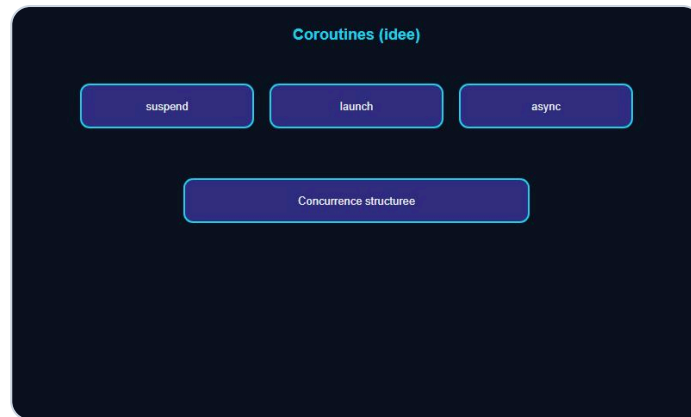
by lazy

```
val config by lazy { loadConfig() }
```

A retenir

- Delegation = reutiliser sans heritage lourd.

Coroutines (idée)



suspend, launch, concurrence structuree.

Les **coroutines** gèrent l'asynchrone sans callback hell.

```
suspend fun charger(): String {  
    delay(100)  
    return "ok"  
}
```

Concepts

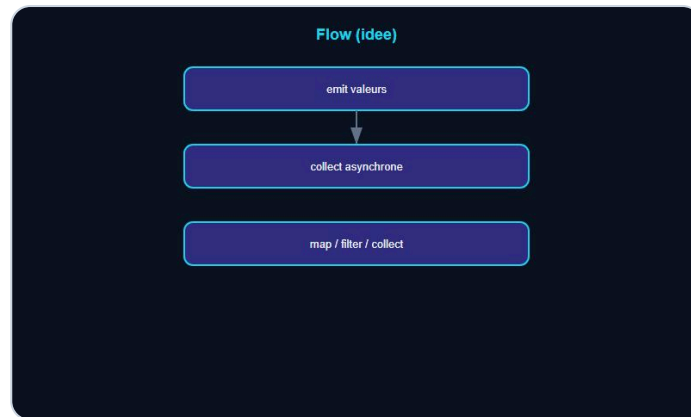
- **suspend** : peut se mettre en pause.
- **launch** / **async** : démarrer du travail.
- **Scope** : qui annule quoi.

> **Astuce DanielCraft** - Apprends **viewModelScope** / **coroutineScope** avant les détails Dispatchers.

A retenir

- Coroutine = concurrence structuree (idée).

Flow (idée)



Stream asynchrone Kotlin.

Un **Flow** émet une séquence asynchrone de valeurs.

```
val ticks = flow {  
    for (i in 1..3) {  
        emit(i)  
        delay(50)  
    }  
}
```

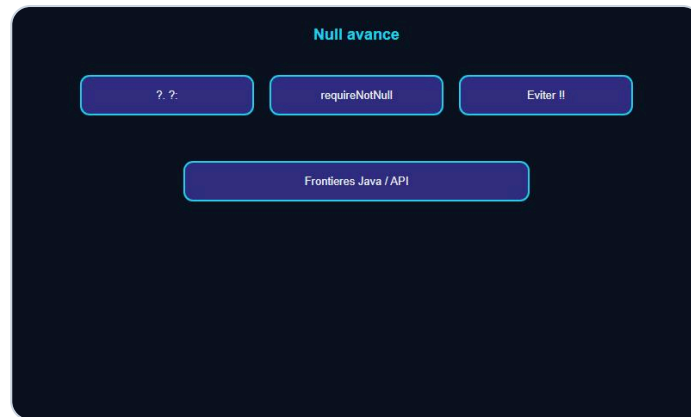
Vs LiveData / callbacks

- Compose bien avec coroutines.
- Opérateurs : `map`, `filter`, `collect`.

A retenir

- Flow = stream asynchrone Kotlin.

Null-safety avancée



Null aux frontieres API.

```
val ville: String? = user?.adresse?.ville
val sure = requireNotNull(id) { "id requis" }
```

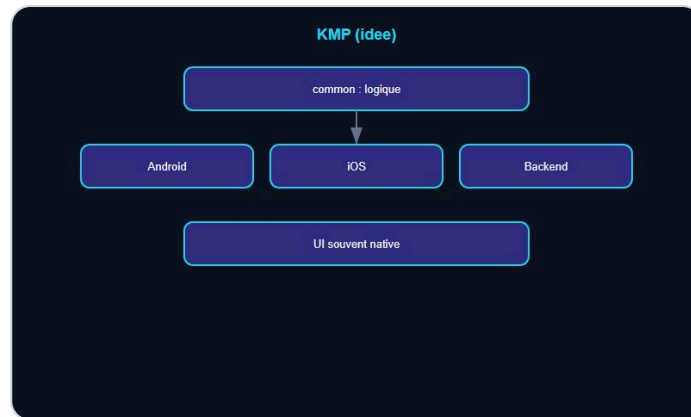
Outils

- `?.` `?:` `!!` (éviter `!!`).
- `takeIf` / `takeUnless`.
- Platform types Java : attention aux null venant du Java.

A retenir

- La surete null se travaille aux frontieres (API, Java).

Kotlin Multiplatform (idee)



Logique partagée, UI native.

Partager de la logique entre Android, iOS, backend...

Idee

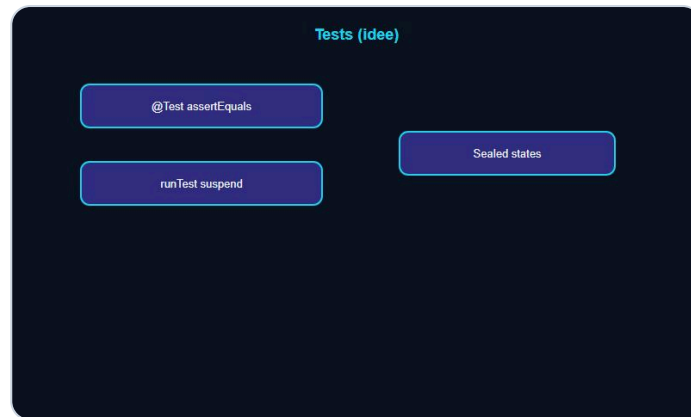
- Module `common` : modèle + règles.
- Modules plateforme : UI / IO spécifiques.

> **Piege** - Ne pas forcer le partage de tout : UI reste souvent native.

A retenir

- KMP = partage du cœur métier, pas magie totale.

Tests (idée)



Tests métier et runTest.

```
@Test
fun addition() {
    assertEquals(4, add(2, 2))
}
```

Coroutines

- `runTest` pour tester le code suspendu.

A retenir

- Tester les règles métier et les sealed states.

Mini-projet : journal d'humeur



Journal d'humeur : sealed + extensions.

Objectif : CLI qui enregistre une humeur par jour.

Exigences

- `sealed class Entree` (Ok / Skip).
- Extensions pour formater la date.
- `groupBy` pour stats par humeur.
- Sauvegarde fichier optionnelle.

Livrables

1. Code Kotlin.
2. 3 tests.
3. when exhaustif sur sealed.

A retenir

- Mini-projet = sealed + collections + extensions.

Ce qu'il faut retenir



Carte inter Kotlin.

Carte inter Kotlin

- Collections avancées, extensions.
- Sealed + when.
- Scope + higher-order.
- Coroutines / Flow (idée), delegation.

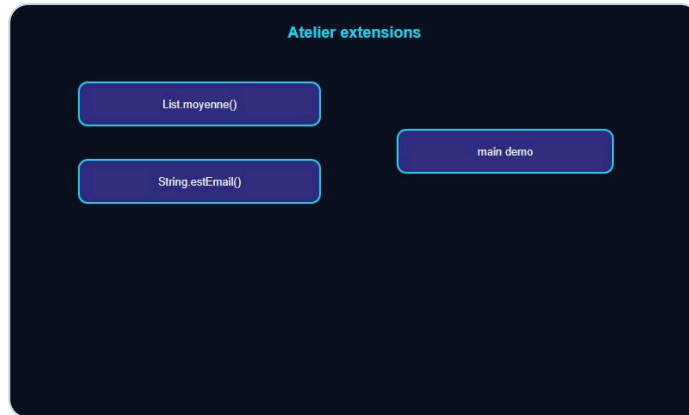
Mindset

- Idiomatique, exhaustif, peu de `!!`.

A retenir

- Kotlin inter = expressif et sur.

Atelier : extensions



Atelier extensions.

Duree : 20 min.

1. `fun List<Int>.moyenne(): Double`.
2. `fun String.estEmailSimple(): Boolean` (contient `@`).
3. Utiliser les deux dans un main.

A retenir

- Extension = API locale lisible.

Atelier : sealed + when



Atelier sealed + when.

Duree : 20 min.

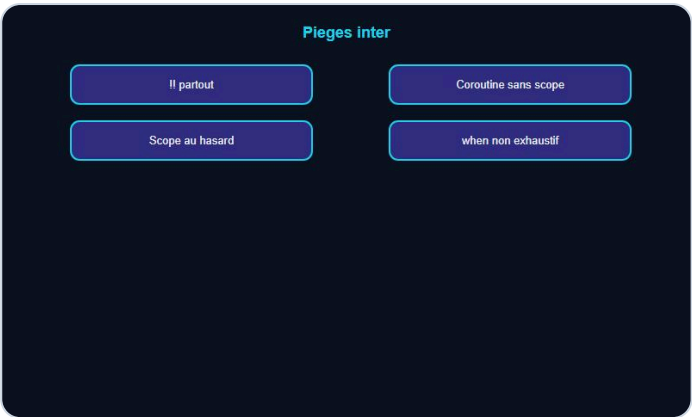
1. `sealed class UiState` : Loading, Data, Error.
2. Fonction `message(state)` avec when exhaustif.
3. Ajouter un 4e cas et corriger la compilation.

A retenir

- Sealed force la mise a jour des when.

CHAPITRE 18

Erreurs classiques



Pieges classiques.

Erreur	Fix
!! partout	?. / requireNotNull
Scope function au hasard	Tableau let/apply
Coroutine sans scope	Scope structure
Java null ignore	Frontieres explicites
when non exhaustif	sealed

A retenir

- Idiomes mal choisis = code Java deguise.

Bonnes pratiques



Style idiomatique.

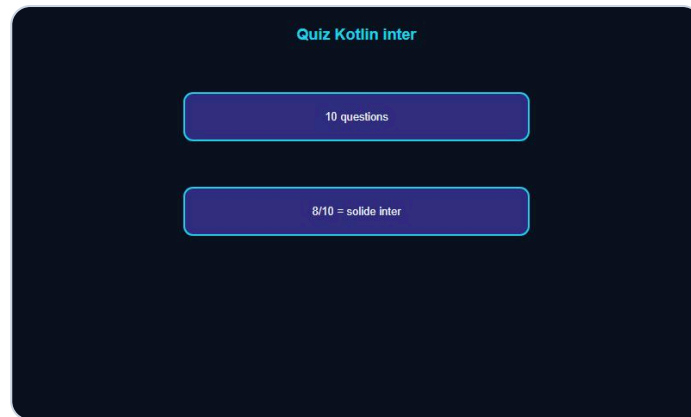
1. Immutable par défaut.
2. Sealed pour états.
3. Extensions ciblées.
4. Peu de `!!`.
5. Tests sur when métier.

A retenir

- Style Kotlin = intention visible.

QUIZ

Quiz Kotlin inter



Quiz Kotlin inter.

1. Interet d'une extension ?
2. Sealed sert a ?
3. Difference apply / let ?
4. Qu'est-ce qu'une higher-order function ?
5. `by lazy` fait quoi ?
6. Idee d'une coroutine ?
7. Flow vs liste ?
8. Danger de `!!` ?
9. KMP partage quoi en priorite ?
10. when exhaustif : avantage ?

Bareme

- 8/10+ : solide pour Android / backend Kotlin.
- Moins : relire sealed + scope.

A retenir

- Quiz = carte mentale.

FINAL

Bravo !



Bravo. Du Kotlin qui respire.

Tu as termine **Kotlin - Intermediaire** : extensions, sealed, scope, coroutines (idee), collections avancees.

Et maintenant ?

- Refactorise un if cascade en sealed + when.
- Remplace un utilitaire statique par une extension.
- Explore Compose ou Ktor avec ces bases.

> **DanielCraft** - Du Kotlin idiomatique, c'est du code qui respire.

A retenir

- Continue : un idiome a la fois, en prod.

Idiomatique. Exhaustif. Sur.